

SELECTED WORKS · 2022 - 2026

Shen Linkun

沈琳昆

Game designer working at the seam between **systems design**,
technical implementation, and **AI-assisted content production**.

Systems Designer at Tencent TiMi Studio Group · B.S. Games,
University of Utah.



I design systems — and build the pipelines that make them producible.

I came into games through design and stayed because of the machinery underneath it. Across four solo/independent titles and two years inside a live mobile MOBA, the thread in my work has been the same: **a design is only as good as the production system that can actually ship it a hundred times over.**

That belief pushed me from writing design documents into writing the tools — behaviour trees, blueprint logic, editor scripts, configuration generators — and, more recently, into **AI-assisted content pipelines**: image generation, image-to-3D, retopology, rigging, and the validation gates that keep generated assets from silently breaking a build.

What I want from a graduate programme is the other half of that loop: the theory, the critical vocabulary, and a studio culture where the question isn't only "**can this be produced?**" but "**what is worth producing?**"

HOW I WORK

- **Framework first, features second.** Structure the system, then fill it.
- **Instrument everything.** If I can't measure the experience, I'm guessing.
- **Automate the repetition, never the judgement.** AI proposes; a human gate approves.

CAPABILITIES

Design

Systems & economy · level design · combat & boss encounters · environmental / looping narrative

Engineering

Unreal Blueprints · behaviour trees · Python tooling · Flask / SQL · CI test gates

Art & Tech Art

Maya · MotionBuilder · ZBrush · Blender · retopology · skinning · PBR · Roblox R15

AI Production

Image generation · image→3D · ComfyUI local deployment · RAG pipelines · multi-model API middleware

CONTENTS

I Game Design — four titles

II Systems Design in Production

III AI-Assisted Art Pipelines

IV Tools & Full-Stack Work

V Original Visual Work

AI Disclosure & Contact

Unreal Engine

Unity

Maya

Blender

ZBrush

Roblox Studio

Python

Flask

ComfyUI

Behaviour Trees

Git

Dream

A first-person psychological horror game about reading a building that refuses to explain itself.

ROLE	ENGINE	DURATION	OUTSOURCED
Solo — design & all Blueprint implementation	Unreal Engine 5	Aug – Dec 2024	Selected art & audio assets only



Fig 1. The verb set is deliberately small — walk, look, and press E. Everything the building says, it says through what it leaves lying on a desk.

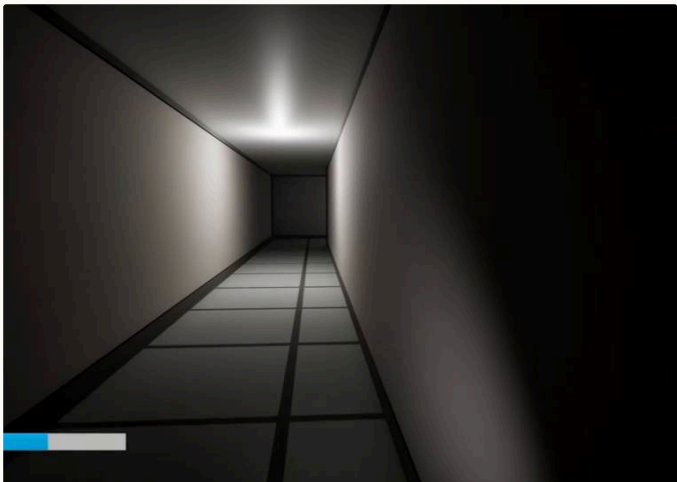
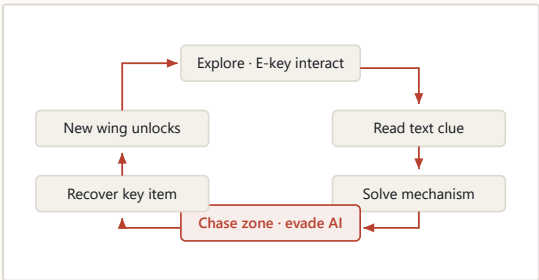


Fig 2. One light source at the vanishing point. The corridor tells you where to go and gives you no reason to want to.

THE CORE LOOP



Exploration is the loop; pursuit is a **rare branch off it**. I deliberately kept the chase scarce — constant threat becomes noise and the player stops feeling it, whereas a threat that has fired once and could fire again turns every ordinary corridor into a **possible** corridor.

DESIGN DECISIONS

- **The threat is rare on purpose.** Pursuit occupies only a small fraction of the running time. Dread does not come from being chased often — it comes from having been chased once and not knowing when it will happen again. Most of the game is the waiting.
- **The AI must be able to lose.** The pursuer disengages once the player exits its detection region. A monster that can never be escaped stops being a threat and becomes a timer.
- **State changes are diegetic.** Triggering a mechanism swaps the score and shifts the practical lights — the player learns the rule of the space without a single line of UI.
- **Resources meter the courage.** A flashlight and a finite stamina bar are the only systems the player manages. Both spend fastest exactly when they are most needed, so caution has a price and panic has a bigger one.
- **Space carries the tension.** Narrow corridors and low key-light do the work that jump-scares would otherwise have to do — fear stays sustained rather than spiked.

PRODUCTION TAKEAWAY

Halfway through I stopped adding features and **refactored the Blueprint layer**, re-grouping scattered logic into explicit categories (Interaction → door / teleport / letter ...). Iteration speed roughly doubled: **framework first, features second**.

Character presentation state machine

Sole owner of the animation state architecture behind a collectible companion system in a live mobile MOBA.

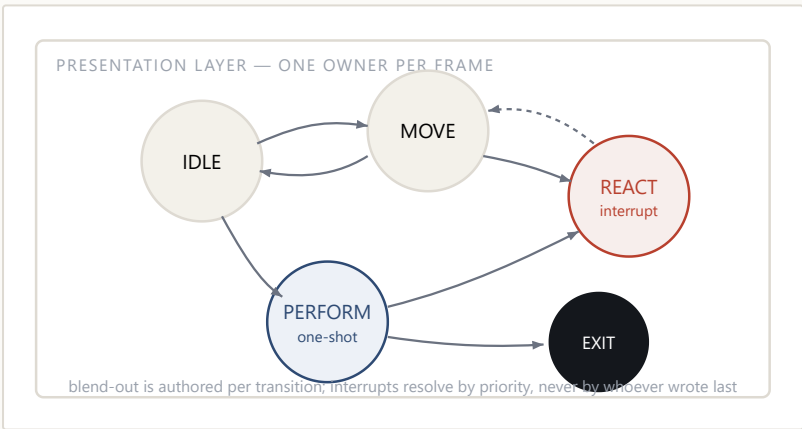
CONTEXT	TITLE	PERIOD	OWNERSHIP
Tencent TiMi Studio Group · Systems Designer	Nation-wide mobile MOBA (live service)	Aug 2025 – present	Sole designer for the state architecture

UNDER NDA No proprietary screenshots, configuration data or engine files are shown. All diagrams in Section II are re-drawn by me at a generic, non-identifying level of abstraction.

THE PROBLEM

Out-of-match companions have to idle, wander, react to the player, respond to taps, and play one-off performance beats — while the player is simultaneously browsing menus, changing outfits and entering queue. The failure mode is not a missing animation; it is a **bad seam**: a companion snapping mid-stride, or freezing because two systems each believed they owned the character that frame.

THE ARCHITECTURE



- **Explicit single ownership.** Exactly one state drives the character each frame; every other system requests rather than sets. This removed the entire class of "two systems fighting" bugs.
- **Transitions are content, not glue.** Each edge has its own authored blend-out, so a walk that is interrupted by a tap decelerates instead of cutting.
- **Interrupts resolve by declared priority.** A reaction can always take the character from idle or move, but never from a one-shot performance mid-beat.

OUTCOME

Out-of-match presentation reads as continuous motion rather than a sequence of clips. Concretely: no more visible pops at the idle→move seam, and reaction beats can be triggered from any menu context without the animation team hand-authoring a case for each one.

WHAT I ACTUALLY DID

- Enumerated every behaviour the feature promised, then cut the state list to the smallest set that could express all of them.
- Wrote the transition matrix — including the illegal transitions, which is where most of the design lives.
- Specified blend timings against the animation team's clips rather than assuming defaults.
- Sat in review with engineering and animation until the seam list was empty.

DESIGN PRINCIPLE CARRIED FORWARD

A state machine is a **contract about who is allowed to lie to the player**. Writing down which transitions are forbidden turned out to be far more valuable than enumerating the permitted ones.

- State machines
- Animation systems
- Cross-discipline spec
- Live service

Gameplay-mode logic: skills, monster AI, boss phases

Adapting a competitive hero roster into an entertainment battle-royale mode — and rebuilding the enemy AI that populates it.

UNDER NDA

Structures below are re-drawn generically; no proprietary logic assets are reproduced.

RE-TUNING SKILLS FOR A DIFFERENT CONTRACT

A competitive MOBA kit is balanced for a 15-minute macro game. Dropped into a fast entertainment mode, the same kit reads as sluggish — long cooldowns and resource costs that exist to create macro decisions instead just create waiting. I worked through the roster adjusting cooldowns, costs and trigger conditions via hero logic graphs and buff triggers so the kits kept their **identity** while matching the new tempo.

THE RULE I WORKED TO

Change the **rhythm**, never the **fantasy**. A player who mains a hero should still recognise their hero; what changes is how often they get to be them.

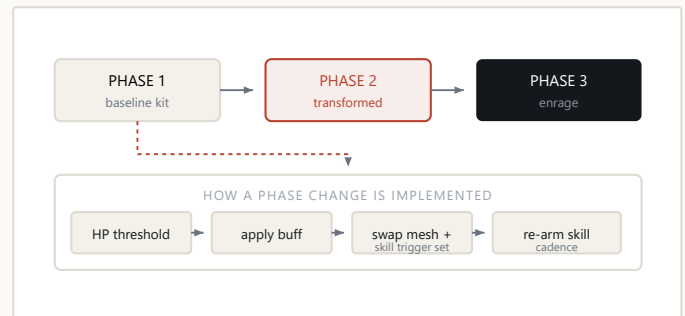
WHERE THE AI ACTUALLY LIVED

A significant finding during this work: the monster AI I was asked to modify was not driven by the visual logic graphs at all, but by **behaviour-tree assets referenced by a path field in the configuration table**. Edits to the graphs had no effect — the graphs were vestigial. Tracing the real execution path before changing anything saved the rest of the schedule.

TRANSFERABLE LESSON

In a decade-old live codebase, **the documented system and the running system are different systems**. I now verify which one I'm editing before I touch it — usually by instrumenting a probe and watching the log, not by reading the code.

BOSS ENCOUNTER: PHASE STATE MACHINE



- **Transformation is a buff, not a new entity.** Driving phase changes through the buff layer keeps threat, targeting and aggro state intact across the transition, so the encounter never resets its social contract with the players fighting it.
- **Reuse the proven AI, replace the form.** Cloning an existing, battle-tested behaviour tree and changing the **presentation** per phase produced a stable boss far faster than hand-authoring a new AI graph — an approach I arrived at after a hand-built version repeatedly deadlocked.
- **Fail loudly.** The debugging discipline that came out of this: a boss that gets stuck in idle is a design bug with an engineering signature, and it is found by reading timestamps, not by intuition.

Behaviour trees

Combat tuning

Boss design

Live debugging

Scaling a mode's content 20 → 110+ with an AI-assisted pipeline

The design problem was never generation speed. It was building a validation gate I could trust.

UNDER NDA

Method described at architecture level only; no configuration data is shown.

20 → 110+

CHARACTERS CONFIGURED FOR THE MODE

≈ 7×

THROUGHPUT VS. MANUAL AUTHORING

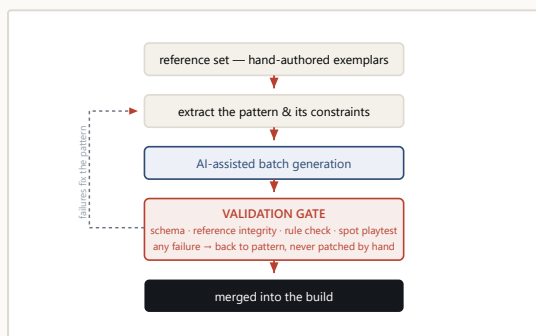
4-stage

AUTOMATED VALIDATION GATE

0

GENERATED ENTRIES SHIPPED UNREVIEWED

PIPELINE ARCHITECTURE



THE ACTUAL INSIGHT

Generation was the cheap part. The expensive part was answering **"how do I know it isn't wrong?"** at a scale where I can no longer read every entry. So the design work went into the gate: a machine-checkable definition of what "correct" means for one entry, applied to all of them.

DESIGN RULES THAT MADE IT WORK

- **Never hand-patch a generated artefact.** A one-off fix hides a systematic defect; the failure has to be pushed back into the pattern so it is fixed 110 times.
- **Encode the traps.** Every production landmine I hit (a path field that silently needs a file extension, an index that must equal its own position, a byte-order marker that crashes a loader) went into a written constraint list the pipeline checks against.
- **Keep a human at the end.** The gate rejects; only I approve. Nothing generated reached the build without review.

WHY THIS IS A DESIGN SKILL, NOT A TOOLING SKILL

Defining the validation gate **is** writing down what the design actually requires — most of what a spec leaves implicit becomes explicit the moment a machine has to check it. The pipeline was, in effect, the most rigorous design document I have ever written.

Content scaling

Validation design

Python tooling

Human-in-the-loop

A character art pipeline for Roblox

Concept → design sheet → modular parts → generated 3D → R15 rig → in-engine. Built as a repeatable route, not a one-off character.

ROLE	TOOLCHAIN	OUTPUT	DOCUMENTED
Pipeline design & end-to-end execution	Image gen · image→3D · Maya · Roblox Studio	Rigged, skinned, engine-ready R15 character	Published as a public process write-up



Step 1 · Concept. Silhouette and palette locked before any 3D exists.

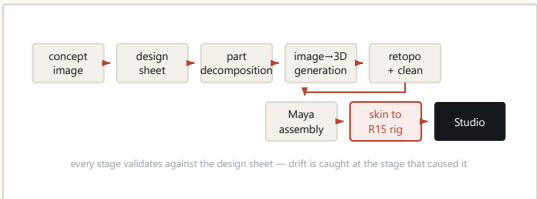


Step 2 · Design sheet. The single source of truth every later stage is checked against.



Step 3 · Orthographic front. Clean, unposed, generation-ready.

THE PIPELINE



The reason to formalise this as a pipeline rather than "make a character" is that the hard problem in generative production is **consistency across stages**. Each handoff is an opportunity for the model to reinterpret the design; putting an explicit reference artefact at the top and checking every stage against it is what keeps stage five looking like stage one.

WHY MODULAR PARTS

Generating a whole dressed character in one pass gives you an un-editable blob. Decomposing into **hair / garment / accessory / footwear** and generating each separately produces parts that can be swapped, re-skinned and re-used across characters — which is the difference between an asset and a wardrobe.

CONSTRAINTS THAT SHAPED IT

- Roblox R15 is a fixed skeleton — the art has to come to the rig, never the reverse.
- Custom characters bind as **accessories** rather than custom rigs, which keeps every stock animation working for free.
- Poly and texture budgets are strict, so retopology is a required stage, not an optimisation.

Pipeline design

Image→3D

Maya

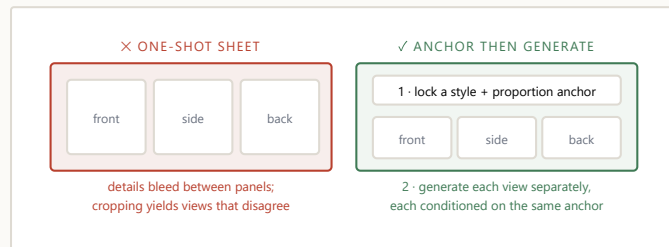
Roblox R15

Tech art

Anchor-then-generate: a method for consistent orthographic views

Reference sheets are the input to every image-to-3D tool. Getting three views that actually agree is the whole problem.

THE FAILURE I STARTED FROM



Asking a model for a three-view sheet in one image is the obvious approach and it fails reliably: adjacent panels contaminate each other, and once you crop them the front and side no longer describe the same character. **Anchoring style and proportion first, then generating each view as its own conditioned pass**, produces views that a downstream 3D generator can actually reconcile.

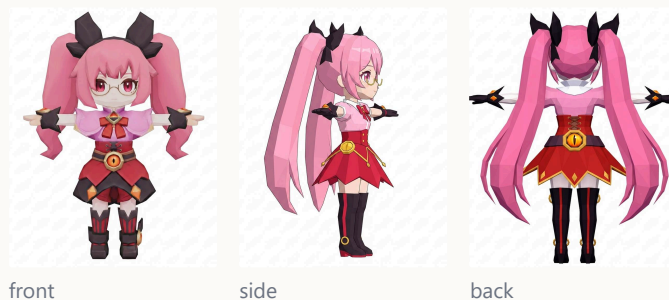


Fig 1. Three independently generated T-pose views that stay in agreement — same proportions, same palette, same costume logic.

REUSABLE BASE BODIES

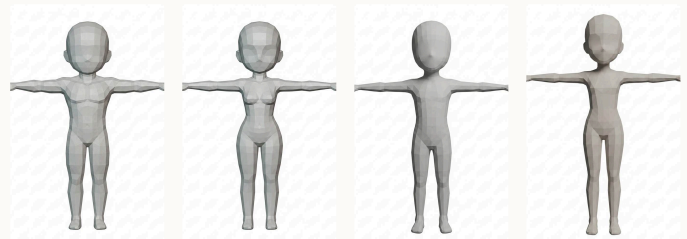


Fig 2. A four-archetype base-body set (adult male / adult female / youth / child) generated once and reused as the proportion anchor for every subsequent character. Style consistency stops being a per-character negotiation and becomes a property of the library.

PART-LEVEL GENERATION



Fig 3. Each garment component generated in its own three views against the shared anchor, so parts remain mix-and-matchable.

WHY THIS MATTERS BEYOND ART

Everything difficult about generative production is a **consistency** problem, not a quality problem. Any model can produce one striking image; a production needs a hundred that belong to the same world. The method above is my answer to that, and it is the question I most want to keep working on academically.

Generative workflow design

Style consistency

Asset libraries

An internal tournament & prediction platform

Designed, built, tested and operated end to end. Ran a full competitive season with a real user base.

ROLE	STACK	SCOPE	STATUS
Solo — product, design, full-stack, ops	Python / Flask · SQL · server-rendered front end	Schedule · virtual-currency prediction · ledger · admin · analytics	Shipped; ran one full season in production

106	70	6.26	0
ACTIVE USERS	SMOKE TESTS GATING DEPLOYS	AVG. ACTIVE PAGES / USER	LEDGER DISCREPANCIES

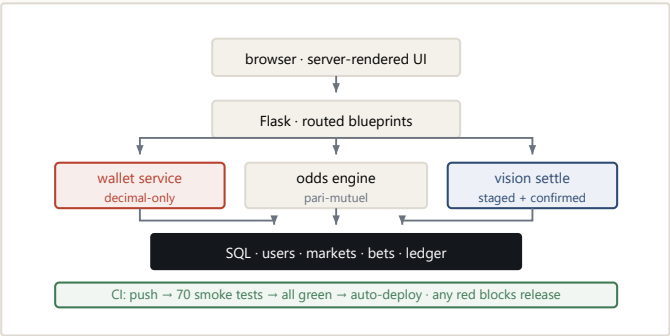
THREE DECISIONS I WOULD DEFEND

- **Money is never a float.** All amounts stored as strings and computed as decimals, behind a single wallet service that every balance change must pass through, with a signed ledger entry per movement. The system is auditable by construction rather than by discipline.
- **The deploy gate is the quality bar.** 70 smoke tests run in CI on every push; the deploy only fires if all pass. Not "run the tests" — **red means it does not ship.**
- **AI proposes, a human commits.** To speed up settlement I added a vision model that reads result screenshots. Its output goes to a staging buffer and a confirmation dialog — **the model never writes to the database.** Low confidence returns "unknown" and settles nothing.

DESIGN LESSON

Real users double-click. Handling that took three layers — front-end submit locking, a server-side duplicate window, and a database uniqueness constraint as the final guarantee. A design that only works when the user behaves is not a design.

ARCHITECTURE



THE ANALYTICS LOOP

I designed the event-tracking taxonomy alongside the features, so behaviour data existed from day one. It showed which pages users actually reached (averaging 6.26 active pages each) and drove the next round of navigation and UI changes. **Instrumenting at design time rather than after launch** is the single practice I would carry into any product I work on.

- Full-stack
- Systems economy
- CI test gates
- Analytics design
- Human-in-the-loop AI

Second Brain — a self-organising knowledge system

A personal research project: a retrieval system that keeps learning, de-duplicating and correcting itself, built as a graph rather than a list.

79

ATOMIC MEMORIES

128

SEMANTIC CONNECTIONS

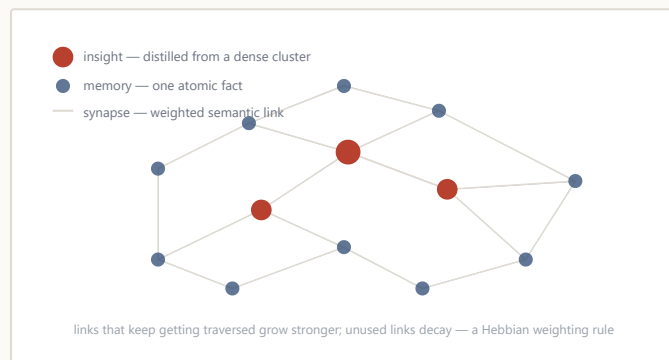
16

HIGHER-ORDER INSIGHTS

3-tier

RETRIEVAL CASCADE

STRUCTURE



Flat vector search returns things that are **similar**. Building explicit connections between memories, then distilling densely-connected clusters into higher-order insights, lets the system return things that are **relevant but not obviously similar** — which is where most useful retrieval actually lives.

TWO IDEAS I AM PROUD OF

1 · A QUARANTINE BEFORE THE LIBRARY

Anything newly acquired lands in a **staging area**, not the main store. It is promoted only after passing a quality gate — length, source credibility, and near-duplicate similarity against existing memories. Most systems optimise retrieval; almost none defend **ingestion**. A knowledge base degrades from what you let in.

2 · A RETRIEVAL CASCADE, CHEAPEST FIRST

Queries try the existing memory store, then a local document sweep, and only then reach the open web. Most questions are answered before the third tier — which saves time, cost, and the temptation to trust a stranger's page over your own recorded experience.

WHY A DESIGNER BUILT THIS

It started as a practical problem — hundreds of scattered production notes I could no longer search — but the interesting part turned out to be an interface question: **what should a system remember on your behalf, and how does it show you that it might be wrong?** That is the research question I would most like to pursue formally.

Retrieval systems

Knowledge graphs

Data visualisation

Python / SQL

Chinese Dreamcore

A personal image series about the interiors of a 1990s–2000s Chinese childhood, made as environment research for an unannounced narrative project.



Fig 1. Twenty locations from one remembered world — bedroom, classroom, computer room, internet café, arcade, station, courtyard, KTV room, street kiosk, stairwell. Generated by me using a controlled prompt-and-reference workflow; see the AI disclosure on page 20.

THE DESIGN PROBLEM: A SERIES, NOT A PICTURE



Before. Generic output — plausible, but belongs to no particular world.



After. Same room, conditioned on a locked style reference.

Twenty images that each look good but do not look like **the same place** are worthless as environment research. The entire craft here is holding a world constant across a set: fixed lens language, fixed time of day, a restricted palette, and a locked reference that every generation is conditioned against. It is the same consistency problem as the character pipeline in Section III, in a different medium.

WHY THIS SUBJECT

Dreamcore usually borrows a Western vocabulary — motel corridors, liminal pools. I wanted to know whether the same uncanny register could be built from **specifically Chinese domestic memory**: the tiled entryway, the exercise-book-covered desk, the courtyard at dusk. The interesting finding is that the unease does not come from distortion at all. It comes from **emptiness plus accuracy** — a room rendered correctly, with the person who should be in it missing.

WHERE IT IS GOING

This set is environment research for a narrative project in the lineage of **Dream** and **0423** — a game about a house you have to read. The images are the art-direction bible, not the final art.

Art direction

Environment design

Series consistency

Generative imaging

Disclosure, credits & contact

GENERATIVE-AI DISCLOSURE

Declared in full, in line with the programme's request.

- **Section III — AI-assisted art pipelines** — generative image and image-to-3D models were used as production tools, which is the explicit subject of that work. Concept direction, part decomposition, retopology, assembly, rigging, weight painting and all in-engine work are mine.
- **Section V — Chinese Dreamcore** — images were generated by me through a controlled prompt-and-reference workflow that I designed, iterated and curated. They are presented as generated images, not as photographs, paintings or hand-rendered art.
- **Section II — content scaling** — AI-assisted generation was used to author configuration data behind a validation gate I designed. No generated entry shipped without human review.
- **Section I — game projects** — no generative AI was used. Design, level construction, Blueprint logic, behaviour trees and animation are hand-authored.
- **This document** — written by me; layout is hand-coded.

CONFIDENTIALITY

Section II describes work carried out at Tencent TiMi Studio Group and is covered by a non-disclosure agreement. **No proprietary screenshots, configuration data, source assets or unreleased content appear anywhere in this portfolio.** All diagrams in that section were re-drawn by me at a generic level of abstraction that conveys the design reasoning without disclosing the product. I would rather present my industry work honestly and incompletely than present it improperly.

CREDITS & SCOPE OF AUTHORSHIP

- **Dream · 0423 · Jim's Adventure** — solo projects; all design and implementation mine, with selected art and audio assets sourced or outsourced.
- **Clay Beats** — team capstone; my ownership was animation production and core interaction design. The title is credited to the full team.
- Industry work (Section II) — collaborative by nature; my specific contribution is stated on each page.
- Everything in Sections III–V is individually authored.

LINKS

Art pipeline write-up (illustrated):

flypigkun.github.io/roblox-art-pipeline

Clay Beats on Steam:

store.steampowered.com/app/3492920

Gameplay reel:

[TODO — paste your YouTube / Vimeo URL here]

CONTACT

Shen Linkun 沈琳昆

ukeming@icloud.com

B.S. Games, University of Utah · Systems Designer,
Tencent TiMi Studio Group

Thank you for reading. Every project here is the same question asked in a different medium: what has to be true about a production system before an idea can survive contact with it?